The Neurotron, a Basic Neural Computing Unit to Implement Sequence Memory

Hugo Pristauz, Walter Eder, Willy Ottinger – Nov. 2023 (Draft v0.5)

Abstract

We introduce the Neurotron*, a basic neural computing unit to build hierarchical temporal sequence memory as a basis for machine intelligence. Sequence memory is capable of learning sequences, and to predict the next item, when only part of a sequence is presented. We also demonstrate that a cluster of Neurotrons can process spatial pattern separation by* sparsifying *and orthogonalization of spatial input patterns. Further we construct a Neurotron cluster to operate as self-learning temporal sequence memory.*

The basic Neurotron*, with the absence of any learning mechanism, is a purely* digital *computing unit. It can be augmented with a learning mechanism based on analog valued synaptic permanences, which represent the development state of the modeled synapses. Such quasi-digital approach for an elementary neural computing unit in combination with a strict application of the fundamental paradigm of sparse pattern representations leads to highly efficient algorithms with fast execution time and low memory needs.*

The Neurotron implements a micro-circuit built from a set of elementary building blocks: terminals, dynamic blocks and gate logic, while terminals are comprising detectors and permanence-units. These building blocks are used in the Neurotron architecture but can be used to compose other abstractions of neural microcircuits. The introduction of universal digital pulse units to mimic spiking and plateau dynamics might be useful for future digital modelling of neural microcircuit abstractions.

The Neurotron follows truly local asynchronous principles, synchroneity is triggered by change of input patterns. Local means, that Neurotrons as basic functional units cannot directly access states of other Neurotrons, except the activation state (the Neurotron's output) , which mimics an action potential of a biological neuron's soma distributed via its axon.

Finally, we benchmarked the performance of a Neurotron based sequence memory layer with the performance of transformer based neural network alternatives, when both systems predict the continuation of text sequences from "Tiny Shakespeare".

Introduction

The neuron model used in most artificial neural networks is known as the perceptron [1], proposed by Frank Rosenblatt in 1957. It has few synapses and no modelling of dendrites (figure 1/A). The perceptron attempts to model the proximal area of the neuron and represents de-facto a mapping of input information arriving at the proximal synapses to the neuron's *output.* There is evidence that the proximal synapses, those closest to the soma (cell body), have a large effect on the likelihood of the cell generating an *action potential,* which is bursting along the neuron's axon to synapses of other neurons of the network. Notably the perceptron model is a pure mapping model without presence of any memory.

In contrast, an excitation of a distal (non-proximal) synapse has little effect at the soma (cell body). For this reason, it was hard to understand how the thousands of distal synapses can play a role for the cell's responses [2].

After development of advanced research methods [3], however, researchers could unlock the secret of internal dendritic NMDA (N-methyl-D-aspartate) spikes. Caused by synaptic excitation within spatial and temporal neighborhood such NMDA spikes lead to a depolarization of the soma, which further enables the neuron to fire earlier than neighbor neurons with comparable proximal excitation. This gives such neuron the benefit to inhibit "competing" neurons.

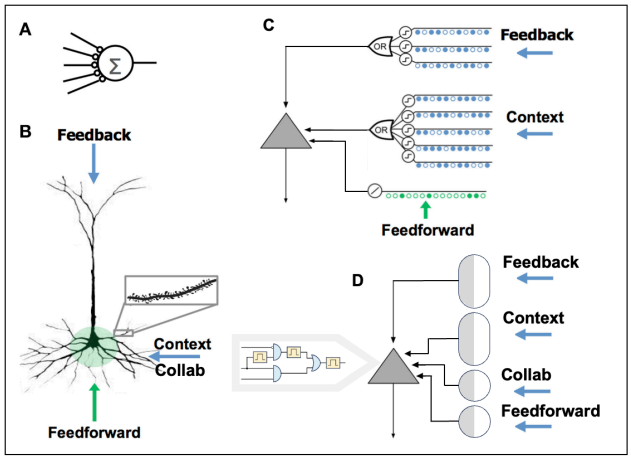

Figure 1: comparison of neuron models

Since NMDA spikes have a much longer duration than the action potential spikes of soma and axon, this mechanism acts like processing memory (based on a depolarized soma state), which is comparable to register memory in microprocessors. In addition to the (perceptron like) mapping functionality of the neuron, the availability of memory in neural models offers capabilities like *prediction* and *bursting*, introduced in [4].
Such and other findings make it evident that biological neurons are very complex systems with complex sub-functionalities in the soma, axon, trunk and dendritic segments. A comprehensive characterization splits neurons into compartments, where each has its own computing capabilities involving spatio/temporal behavior of various ion concentrations as well as electrical quantities. As studies have shown the information flow between soma and other compartments can be bidirectional (figure 2, [9]). Some of this approaches found in the literature are shown in figures 1,2,3.

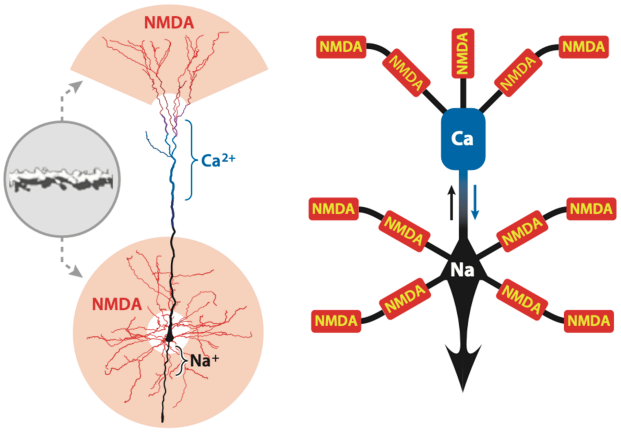

Figure 2: Multi compartment model respecting local computations in different dendritic segments [9].

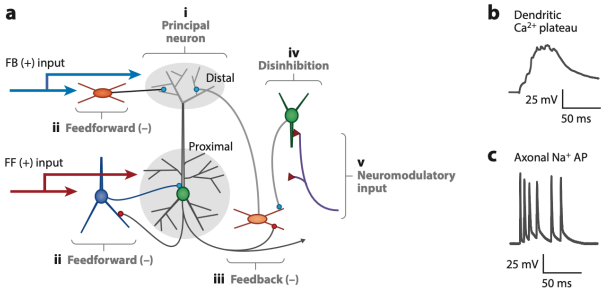

Figure 3: Cortical microcircuit showing different input kinds applied to a principal (pyramidal) neuron [6]

Such detailed models without further abstraction (often described as analog spiking models [6,7]) make it, however, hard to understand behavior in higher levels, like explanations how the brain might model sequence memory. Thus, understanding of any given neuron requires the right level of abstraction [2].

Inspired by such findings regarding distal dendrite functionality a group around Jeff Hawkins developed the HTM approach [4,5], which we consider a powerful conceptual abstraction of an enhanced neural layer functionality serving as an abstract building block for truly intelligent algorithms, which can run efficiently on computers. It incorporates the following concepts:

- The fundamental paradigm of sparse pattern representation to implement pattern separation [4,5,6]
- *Quasi-digital processing*: most of the signals have binary values (0 or 1), except permanence values and synaptic thresholds (analog values in the interval [0,1]).
- *Binary synaptic weights*: causing a synapse to behave as unconnected (weight 0) or connected (weight 1). A weight is controlled by the analog *permanence state* of the synapse (range [0,1]), causing the weight to take value 1 if the *permanence* exceeds a synaptic threshold, and 0 otherwise.
- *Collaborations:* a collaborative group of neurons (in [4] called *minicolumn*) with a mechanism for "*voluntary neuron*" selection, which assigns a neuron with ownership to take over co-representation of an unexpected pattern.
- *Predictive neuron states:* enable those neurons of a collaboration to fire, which are expected to do so on base of the processed input pattern, while other neurons of the collaboration stay quiet.
- *Burst:* a state being activated during presentation of unexpected patterns. Bursting causes all neurons of a collaboration to vote for ownership of pattern co-representation.
- *Local learning:* the learning mechanism is Hebbian like. In contrast to Hebbian learning where weight adjustment is correlated with the product of pre-synaptic input and neuron output, HTM neurons are only empowered for learning, if they were able to make a correct prediction.

There are some notable remarks: The quasi-digital nature of the algorithms in combination with sparse pattern representations allows very fast and efficient processing on digital computing hardware. The capability of *bursting* in case of a new detected pattern increases the probability of finding voluntary neurons to take over co-representation ownership for unexpected patterns, which contributes to learning efficiency.

The HTM learning rule, where synapses are only credited for learning, when a neuron made a correct prediction, provides strong learning focus on neurons with actual involvement in the prediction process. It helps to avoid ruinous overwriting of well-established memory contents, which is a key challenge in the formulation of suitable synaptic plasticity hypothesis [8].

The HTM algorithm presented in [4] is formulated for a neuron-layer comprising separated *minicolumns* as an array of "HTM-neurons". The concept implies that each neuron is part of exactly one *minicolumn*. We replaced the concept of *minicolumn* by the concept of *collaboration*, where Neurotrons belong to one or more collaborative groups,

which, in the biological equivalent, is more likely than a strict organization in terms of separate *minicolumns*.

Also, the description in [4] leaves it fuzzy how the algorithm might be implemented on the granularity of neurons. Such approach is reasonable, when the intention is to provide building blocks for higher level cognitive functionality. When we designed the *Neurotron*, however, we wondered how the HTM algorithm could be broken down to the granularity of neurons.

While in [4] the basic computing unit is the *minicolumn*, we are going to introduce an even more elementary computing unit, the *Neurotron*, which can be used to build either *minicolumns* on the next higher level, or in a more general form to build *collaborations* as part of a cluster layer. Alternatively, we expect that *Neurotrons* can be the building blocks for higher level functional units like the *Spatial Pooler* consulted in the HTM approach, or an abstract model of the pattern separation functionality of the dentate gyrus modeled in [6], although both applications have not yet been studied in detail.

The Neurotron in a Nutshell

Now we are going to introduce the *Neurotron* in a top-down approach. There are four atomic functional units employed by the Neurotron:

- *Detectors*: representing the dendritic capability of coincidence detection, which is a pure binary mapping functionality (without memory).
- *Dynamic Blocks:* digital abstractions of dynamic behavior of neuronal compartments (soma, dendritic segments. We use a unified parametrizable *pulse unit* to implement a dynamic block.
- *Gate Logic:* a combination of elementary logic functions like AND, OR and NOT with pure mapping functionality (excluding memory functions).
- *Permanences:* analog valued, matrix organized memory representing the development state of synapses. Each row of a permanence matrix corresponds to synapses of a dendritic segment.
-

The *Basic Neurotron*, which does not support a learning mechanism, comprises only three of the listed atomic units, which have pure digital character (*detector, dynamic block and gate logic*). The *augmented* or *general Neurotron* employs also the *permanence* unit, which has analog operation. Thus, we call a (*general*) *Neurotron* network *quasi-digital*.

A *detector, optionally* augmented with *permanences* is called a *terminal* of the *Neurotron*. A terminal represents a compartment of a biological neuron where axons of other neurons arrive at neighbored synapses. Considering the permanence state as constant in a given time slice, a *terminal*, notably, has pure mapping functionality, i.e., the coincidence detection functionality of the underlying *detector*.

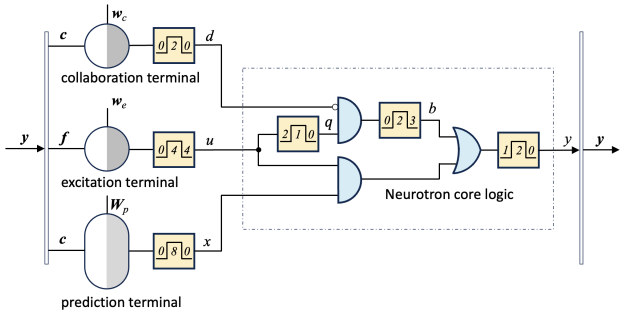

Figure 4: schematics of the *Basic Neurotron*

Before explaining the details of the employed atomic building blocks let us have a look on the overall schematics of the *Basic Neurotron* (figure 4). Notably, the aim is not to present the *Neurotron* as a proven model of a given neural microcircuit, rather the *Neurotron* is formulated as a *generic computation unit inspired by neural mechanisms*, which is capable to implement high level functions like *pooling* and self-learning sequence memory.

More specifically we claim that a cluster of *N* properly parametrized *Basic Neurotrons* according to figure 2

- are able to perform a *pooling* function, i.e., a mapping of non-sparse binary patterns into sparse binary patterns maintaining sematic relationships
- are able to implement a sequence memory, where each *Neurotron* operates *asynchronously* and *locally* with only access to the outputs of other Neurons in the collection, but no access to other internal state variables of other neurons.
- Are able to perform a state observation from the Neurotron outputs, i.e., are able to reconstruct if any Neurotron in a given group is in a prediction state in order to suppress a *burst condition* (as it is described in [4]).

Furthermore, the *Basic Neurotron* algorithm can be augmented with a true local (Hebbian style) learning mechanism, operating in a quasi-digital mode, which implies to augment some of the terminals with adaptable *permanences*.

The Neurotron operates on changing patterns of two input vectors, *feedforward vector* $\boldsymbol{f} \in B^M$ (B denotes the binary set $\{0,1\}$), which encodes the sequence tokens to be processed, and *context vector* $\boldsymbol{c} \in B^N$, which collects the outputs of all *Neurotrons* in a processing cluster, including the *Neurotron* itself and those of the collaborative groups. Formally $\boldsymbol{f}$ and $\boldsymbol{c}$ are composed to *output vector* $\boldsymbol{y}$, which represents the outputs of *all Neurotrons* being involved in the processing scheme. Since the Neurotron output y is part of *output vector* $\boldsymbol{y}$, it will update the corresponding element y_k of $\boldsymbol{y} = (\ldots, y_k, \ldots)$ continuously.

Dynamic Blocks (Pulse Units)

Dynamic blocks are used in the Neurotron model to represent (binary) state. In neurobiology it has been observed that action potentials are aggregated by neuronal compartment to potentially exhibit spikes or plateau potentials, which have leading delays, plateau effects and relaxation phases, during which a reactivation is temporarily suppressed. We introduce a parametrizable time discretized digital *pulse unit* (see section "materials and methods" for a detailed description) as a universal dynamic block denoted by

output sequence = Pulse(input sequence, lag, duty, relax)

where the integer parameter *lag* models the leading delay (in discrete time units) during which an integration of the (binary) input signals happen. Once the integrated value reaches a threshold, which equals the lag parameter, the *lag state* of the pulse unit transitions into the *duty state,* with activated output (binary 1) over a number of discrete time units corresponding to the *duty* parameter. In the simple case (third parameter *relax* is zero), the output falls back to zero after the duty period (figure 3b,d), except the pulse unit is being retriggered (figure 3c). In the absence of bouncing input, the pulse unit behaves like a delayed re-triggerable mono flop.

Figure 5e shows the behavior of input bouncing (lag=2, duty=3, relax=0). The requirement of 3 (lag+1) consecutive one-inputs is initially not achieved, thus the integrator falls back from intermediate level 2 to level 1 (not shown), which needs two additional one-inputs to reach the integration threshold, causing finally a duty state transition to hold the output at level one (over 3 duty steps), before flipping back to zero (figure 5e). Thus, the integrator at the input side of the pulse unit performs a debouncing function, which is important for the Neurotron during periods where the patterns applied to the Neuron's detectors are floating.

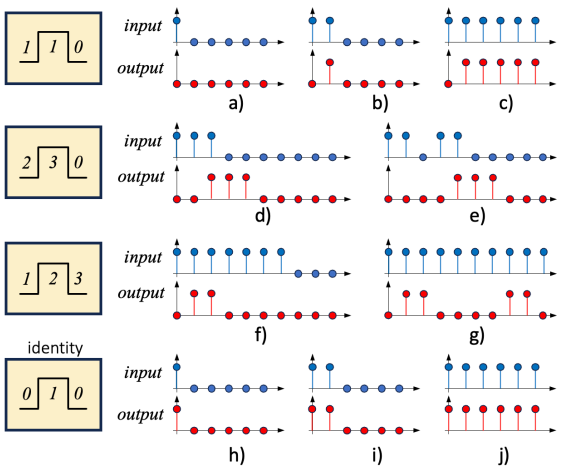

Figure 5: dynamic block represented as pulse unit in action

In figure 5f,g a pulse unit with activated relaxation (delay=1, duty=2 and relax=3) is shown in action. After each duty period (length 2) a relaxation period of length 3 must follow, during which the output is kept zero. Figure 5g also demonstrates, that re-triggering is ignored in case of a non-zero relax parameter.

Figures 5h,i,j demonstrate that a parameter choice of lag=0, duty=1 relax=0 leads to an identity block which passes an input sequence unchanged to the output. This means that dynamic blocks can be deactivated just by proper choice of parameters.

A finite state machine implementing a *pulse unit* to serve as dynamic block is shown in figure 6. At the input side (input *u*) it has a discrete integrator which increments counter x for u = 1, and decrements counter x for u = 0, maintaining the limitation $0 \le x \le l$. This integrator also debounces the input for lag time $l > 0$. Besides of counter x, which determines the end of the LAG phase, there is another counter c, to control the duration of the DUTY or RELAX phase. Depending on the values of u, x and c two auxiliary variables y' and c' are calculated, which, together with machine state $s \in \{\text{LAG, DUTY, RELAX}\}$, control the actions of the state machine according to the state transition diagram in figure 6, which also set the output *y* properly.

The state machine of figure 6 is a synthetical construct to meet given input/output behavior shown in figure 5 and has no direct inspiration from neurobiological structures, except to mimic neurobiological pulse dynamics with initial delay (lag), duty phase (plateau) and an optional relaxation phase during which input signals are temporarily ignored.

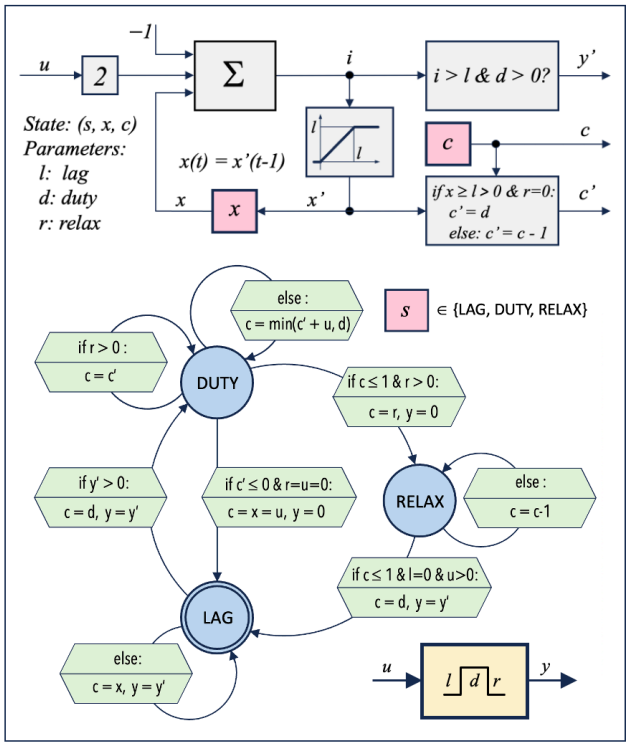

Figure 6: finite state machine implementing a dynamic block represented by a pulse unit

Detectors

Detectors have the role of coincidence detection, which is very similar to the role of digital decoding in electronic circuits to select subunits (like a processor register or I/O port) by application of a proper address at the decoder's input. If such subunit should be selected by the binary address $\boldsymbol{a} = [1,0,1,0]$ according to a given decoder input $\boldsymbol{v} = [v_1,v_2,v_3,v_4]$, then the decoder operation is a mapping

$$s = v_1 \wedge \neg\, v_2 \wedge v_3 \wedge \neg\, v_4 .$$

This decoding scheme discriminates reliably an intended address pattern **a** from and any other possible pattern. Such approach can be adopted for the detection of sparse patterns. The 1-norm of a binary vector x $\in B^n$ is defined as *(1)*

$$||\boldsymbol{x}||_1 = ||(x_1 , x_2 , \ldots x_n)||_1 := \Sigma_i |x_i| = \Sigma_i x_i = \langle \boldsymbol{x}|\boldsymbol{x}\rangle , \quad (1)$$

with $\langle \boldsymbol{a}|\boldsymbol{b}\rangle$ denoting scalar product. We write $\boldsymbol{x} \leq \boldsymbol{w}$ *iff* (if and only if) $||\boldsymbol{x}||_1 \leq ||\boldsymbol{w}||_1$, and we call $\langle \boldsymbol{w}|\boldsymbol{x}\rangle$ the overlap of the two vectors **w** and **x**. Given a weight vector $\boldsymbol{w} \in B^n$ and a threshold Θ we define a *detection target* as the set

$$D(\boldsymbol{w}, \Theta) := \{\boldsymbol{x} \in B^l \,|\, \boldsymbol{x} \leq \boldsymbol{w} \wedge \langle \boldsymbol{w}|\boldsymbol{x}\rangle \geq \Theta \} \quad (2)$$

In other words, the *detection target* $D(\boldsymbol{w}, \Theta)$ is the set comprising all binary vectors $\boldsymbol{x} \leq \boldsymbol{w}$ (number of non-zero bits of $\boldsymbol{x}$ must not exceed those of $\boldsymbol{w}$) with overlap $\langle \boldsymbol{w}|\boldsymbol{x}\rangle$ greater than or equal to threshold Θ. A *detection target* $D(\boldsymbol{w}, \Theta)$ induces a map

$$f: B^l \rightarrow B \text{ with } \boldsymbol{x} \mapsto (\boldsymbol{x} \in D(w, \Theta)), \quad (3)$$

i.e., a *detector* triggers (outputs a logical 1) *iff* the input pattern $\boldsymbol{x} \leq \boldsymbol{w}$ is part of its *detection target.* The smaller the threshold Θ is chosen, the bigger is the *detection target (set)*, and the bigger is the chance for a pattern close to the reference pattern of the *target* (given by weight vector $\boldsymbol{w}$) to trigger the detector. An important consequence is that all vectors of a *detection target* have similar semantic meaning. To complete the picture, there are also input vectors $\boldsymbol{x} > \boldsymbol{w}$ outside of a *detection target*, $\boldsymbol{x} \notin D(w, \Theta)$, which can also trigger the detector. Under a paradigm of sparsity, however, such cases are considered as pathological and expected to occur very rarely.

Coming back to the electronic example: if we deal only with sparse patterns $||\boldsymbol{v}||_1 \leq 2$, then choosing $\boldsymbol{w}$ = (1, 0, 1, 0) and $\Theta = 2$ defines a *detection target* $D((1,0,1,0), 2) =$ {(1,0,10)} for a detector. Such detector triggers only for one single input pattern $\boldsymbol{v} = \boldsymbol{w}$, thus it is equivalent to the electronic decoder above. Lowering the threshold to $\Theta = 1$ induces a much bigger *detection target*:

$$D((1,0,1,0), 1) = \{(1,0,1,0), (1,0,0,0), (0,0,1,0), (1,1,0,0), (1,0,0,1), (0,1,1,0), (0,0,1,1)\} ,$$

implying to trigger on additional patterns, which are similar to the reference pattern determined by weight vector $\boldsymbol{w}$.

The functionality introduced by our *detectors* has some similarity with the classic McCulloch-Pitts model of a neuron, introduced in 1943 (figure 6a), when only those inputs are connected to the neuron model which relate to a one in the weight vector (for $\boldsymbol{w}$ = (1,0,1,0) => inputs v_1 and v_3). Our detector be considered as an augmented Mc-Culloch-Pitts model with weight vector $\boldsymbol{w}$ as an additional parameter (figure 6b).

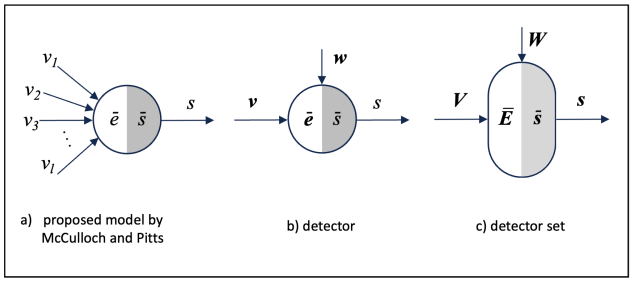

Figure 6: Detector symbols based on the digital neuron model of McCulloch and Pitts in 1943

While the mapping of a McCulloch-Pitts model is described by (1a-b),

$$e = \bar{e}(v_1, v_2, \ldots, v_l) = \Sigma_i v_i \quad (1a)$$
$$s = \bar{s}(e) = (e \geq \Theta) \quad (1b)$$

the detector employs the augmented input/output mapping (2a-d) with ◦ denoting elementwise multiplication.

$$\boldsymbol{v} = (v_1, v_2, \ldots, v_l) \in B^l \quad (2a)$$
$$\boldsymbol{w} := (w_1, w_2, \ldots, w_l) \in B^l \quad (2b)$$
$$\boldsymbol{e} = \bar{e}(\boldsymbol{v}) = \boldsymbol{w} \circ \boldsymbol{v} \in B^l \quad (2c)$$
$$s = \bar{s}(\boldsymbol{e}) = (\Sigma e_i \geq \Theta) \in B^l \quad (2d)$$

Weight vectors are a convenient way to describe a given connection topology. As an example, let us assume a cluster with 10.000 Neurotrons, each having a binary output y_k (k = 1 … 10000), represented as output vector

$$\boldsymbol{y} = (y_1, y_2, \ldots y_{10000}) \in B^{10000}.$$

If we want to model a detector with 3 inputs v_1, v_2, v_3 with the following connection topology

$$y_2 \rightarrow v_1, \quad y_6 \rightarrow v_2, \quad y_8 \rightarrow v_3$$

we would just define a detector mapping from $f: B^{10000} \rightarrow B$ and a weight vector

$$\boldsymbol{w} = (0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, \ldots) \in B^{10000}.$$

A set of *detectors* operating on a set of weight vectors $\boldsymbol{w}_k$ can be combined to a *detector set* parametrized with weight matrix $\boldsymbol{W}$ and operating on a set of patterns represented by

matrix $\boldsymbol{V}$ (figure 6c). Such *detector set* is useful for modelling a set of dendritic segments. It implements the mapping (3a-d).

$$\boldsymbol{V} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_d)^T \quad \in \mathcal{B}^{d \times l} \qquad (3a)$$
$$\boldsymbol{W} := (\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_d)^T \quad \in \mathcal{B}^{d \times l} \qquad (3b)$$
$$\boldsymbol{E} = \bar{E}(\boldsymbol{V}) = \boldsymbol{W} \circ \boldsymbol{V} \quad \in \mathcal{B}^{d \times l} \qquad (3c)$$
$$\boldsymbol{s} = \bar{s}(\boldsymbol{E}) = [\Sigma \boldsymbol{e}_i \geq \Theta] \quad \in \mathcal{B}^{d} \qquad (3d)$$

We call vector e in (2c) the empowerment vector with *(4)* defining the overlap of vectors **v** and $\boldsymbol{w}$,

$$\|\boldsymbol{e}\|_1 = \|\boldsymbol{v} \circ \boldsymbol{w}\|_1 = \langle \boldsymbol{v} | \boldsymbol{w} \rangle = \Sigma_i |e_i| = \Sigma_i e_i \qquad (4)$$

and call *s* the spiking signal, which equals 1 *iff* the overlap $\| \boldsymbol{e} \|_1$ exceeds threshold Θ. We say that a *detector spikes,* if it outputs a non-zero spiking signal, which has the equivalent meaning that the decoder has detected an input pattern *coincidence* with the *detection target*.

In a similar way we call $\boldsymbol{E}$ in (3c) the *empowerment matrix* of the *detector*, and vector $\boldsymbol{s}$ in *(3d)* the *spiking vector*, collecting the *spiking signals* of each *detector* of the set.

Neurotron States and Core Logic

We are going to explain now the *basic states* and the *core logic* of the *Neurotron* (figure 7).

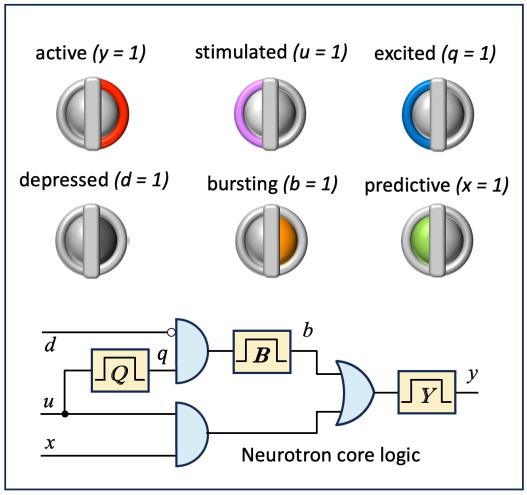

Figure 7: basic states and core logic of the *Neurotron*

The meaning of the *basic states* is as follows.

- *Active (y = 1)*: The *Neurotron* mimics a biological neuron's state in firing condition, i.e., the soma fires an action potential propagating along the axon.
- *Stimulated (u = 1)*: The *Neurotron* mimics a biological neuron's condition, where the aggregation of feedforward signals arriving at proximal synapses is strong enough to enable activation under a predictive condition. If the Neurotron is not predictive, stimulation is not sufficient for Neurotron activation.
- *Excited (q = 1)*: If stimulation is sufficiently long, the *Neurotron* gets excited. After some time in excited state a Neurotron, which is not depressed, starts bursting
- *Depressed (d = 1)*: The Neurotron mimics a biological neuron's condition where some collaborating neurons are trying to prevent the neuron from firing. In a *depressed* state a Neurotron is not able to *burst*.
- *Bursting (b = 1)*: A non-depressed Neurotron in excited state will start bursting after some delay. This in turn causes *activation*, i.e., setting the output signal active.
- *Predictive (p = 1):* The *Neurotron* mimics a biological neuron's condition where sufficient aggregation of action potentials arriving at distal synapses at previous time caused some dendritic segment to evoke an NMDA spike, which depolarized the soma. In such condition the neuron "predicts" an upcoming (feed-forward) stimulation with subsequent excitement and activation.

The logical dependency between states *u, d, x, y, q, b* and y is formulated as the *Neurotron core logic* (see also figure 7), defined as follows:

- A *Neurotron* gets active when it is
  a) either both stimulated and predictive
  b) or excited and not depressed

The formal description of the *Neurotron core logic* is formulated in *(5a-c)*.

$$q = Q(u) \qquad (5a)$$
$$b = B(q \circ not\ d) \qquad (5b)$$
$$y = Y(u \circ x \ \ or\ b) \qquad (5c)$$

See figure 8 for different scenarios following the *Neurotron core logic*

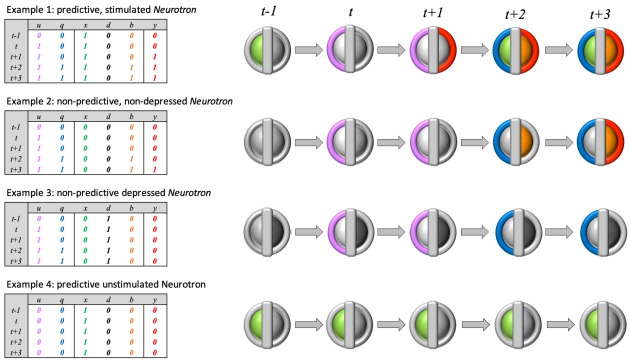

Figure 8: several examples demonstrating the application of the *Neurotron core logic*

Sequence Memory

The following "toy" example demonstrates how a 3-*Neurotron* network can predict the next item, when parts of the

sequence <Sarah loves music> are presented. In this example the words Sarah, loves and music are represented by *none-sparse* feed-forward patterns $\boldsymbol{f}_i \in \mathcal{B}^{10}$ (called *tokens*).

Sarah $\mapsto$ $\boldsymbol{f}_1 = [1, 1, 0, 1, 1, 1, 0, 1, 0, 1]$
loves $\mapsto$ $\boldsymbol{f}_2 = [0, 1, 1, 1, 0, 1, 1, 0, 1, 1]$
music $\mapsto$ $\boldsymbol{f}_3 = [1, 1, 1, 0, 0, 1, 0, 1, 1, 1]$

The first tasks for our 3 *Neurotrons* is to map these feed-forward patterns into sparse orthogonal *stimuli* $\boldsymbol{u}_i \in \mathcal{B}^3$. The following map assigning the basis vectors of $\mathcal{B}^3$ will do such job.

$$\boldsymbol{f}_1 \mapsto \boldsymbol{u}_1 = (1,0,0), \quad \boldsymbol{f}_2 \mapsto \boldsymbol{u}_2 = (0,1,0), \quad \boldsymbol{f}_3 \mapsto \boldsymbol{u}_1 = (0,0,1)$$

From the definition of a *detector*, we know that an input pattern must belong to its *focus set* in order to trigger. Thus, the goal is to find parameters of the excitation terminal (weight vectors $\boldsymbol{w}_{u1}, \boldsymbol{w}_{u1}, \boldsymbol{w}_{u3}$ and thresholds $\Theta_{u1}, \Theta_{u2}, \Theta_{u3}$) to satisfy the following equations:

$$\begin{aligned}
&\boldsymbol{f}_1 \in F(\boldsymbol{w}_{u1}, \Theta_{u1}), \ \boldsymbol{f}_1 \notin F(\boldsymbol{w}_{u2}, \Theta_{u2}), \ \boldsymbol{f}_1 \in F(\boldsymbol{w}_{u3}, \Theta_{u3}) \\
&\boldsymbol{f}_2 \notin F(\boldsymbol{w}_{u1}, \Theta_{u1}), \ \boldsymbol{f}_2 \in F(\boldsymbol{w}_{u2}, \Theta_{u2}), \ \boldsymbol{f}_2 \notin F(\boldsymbol{w}_{u3}, \Theta_{u3}) \\
&\boldsymbol{f}_3 \notin F(\boldsymbol{w}_{u1}, \Theta_{u1}), \ \boldsymbol{f}_3 \notin F(\boldsymbol{w}_{u2}, \Theta_{u2}), \ \boldsymbol{f}_3 \in F(\boldsymbol{w}_{u3}, \Theta_{u3})
\end{aligned}$$

An easy check proves that this works for $\boldsymbol{w}_{ui} = \boldsymbol{f}_i$ and $\Theta_{ui} \in \{6,7\}$, $i = 1..3$. With such parameters the excitation terminals implement the following map, which performs both sparsifying and orthogonalization [6].

Sarah $\mapsto [1, 0, 0]$, loves $\mapsto [0, 1, 0]$, music $\mapsto [0, 0, 1]$ □

In the next step we define the framework conditions, under which our 3 *Neurotrons* are operating: All *Neurotron* outputs y_k are collected by the *context vector* $\boldsymbol{c}$,

$$\boldsymbol{c} = (y_1, y_2, y_3)$$

which, together with *feedforward vector* $\boldsymbol{f}$, forms the *network output vector* $\boldsymbol{y}$.

$$\boldsymbol{y} = (\boldsymbol{c}, \boldsymbol{f}) = (y_1, y_2, y_3, f_1, f_2, \ldots, f_{10})$$

According to figure 4 each Neurotron k decomposes the *network output vector* $\boldsymbol{y}$ into $\boldsymbol{c}$ and $\boldsymbol{f}$ and feeds them accordingly to the Neurotron's *terminals*. After processing the *detection* operation in the *terminals* and running the attached *pulse dynamics*, the *Neurotron* runs its *core logic* in order to processes the new output signal y_k, which is finally updated in the *network output vector* $\boldsymbol{y} = (\ldots, y_k, \ldots)$.

In the next step we define the network structure, including the definition of *collaborations* (collaborative Neurotron groups). For the demonstrations in the current example, we do not need collaborative groups. Thus, we define the *weight vectors* $\boldsymbol{w}_{ci} \in \mathcal{B}^3$ of the *collaboration terminal* as zero vectors.

$$\boldsymbol{w}_{c1} = \boldsymbol{w}_{c2} = \boldsymbol{w}_{c3} = (0,0,0)$$

For the prediction terminal we choose a *detector set* with two *detectors*, requiring the *prediction weight matrices* to be $\boldsymbol{W}_{pi} \in \mathcal{B}^{2\times 3}$, which we setup as follows:

1) when the first item Sarah is presented to the network, the network's *excitation terminals* map token *f1 to stimulus* $\boldsymbol{u} = (1,0,0)$ *which causes Neurotron 1 to fire, causing* $\boldsymbol{c} = \boldsymbol{u}_1 = (1,0,0)$. However, there is nothing for *Neurotron 1* to predict, since Sarah is the first item of the input sequence. Thus, we set

$$\boldsymbol{W}_{p1} = [\,0, 0, 0;\ 0, 0, 0].$$

2) On the other hand, *Neurotron 2* detects itself to become *predictive* by stimulus $\boldsymbol{u}_1 = (1,0,0)$ (encoding Sarah) causing $\boldsymbol{c} = \boldsymbol{u}_1$, since *Neurotron 2* has to fire when stimulus $\boldsymbol{u}_2 = (0,1,0)$ (encoding loves) is presented to the network, , causing $\boldsymbol{c} = \boldsymbol{u}_2$. This is achieved by setting one of the rows of $\boldsymbol{W}_{p2}$ to match $\boldsymbol{u}_2$, e.g.:

$$\boldsymbol{W}_{p2} = [\,\boldsymbol{u}_1;\ \boldsymbol{0}\,] = [\,1, 0, 0;\ 0, 0, 0]$$

3) In a similar way Neurotron 3, encoding music represented by $\boldsymbol{u}_3 = (0,0,1)$, has to get predictive when $\boldsymbol{u}_2 = (0,1,0)$ (encoding loves) is detected, causing $\boldsymbol{c} = \boldsymbol{u}_2$. Thus, we can choose

$$\boldsymbol{W}_{p3} = [\,\boldsymbol{u}_2;\ \boldsymbol{0}\,] = [\,0, 1, 0;\ 0, 0, 0].$$

Remark: It does not matter which *detector* of the *prediction terminal's decoder set* does the prediction. Thus, the choices $\boldsymbol{W}_{p2} = [\,\boldsymbol{0};\ \boldsymbol{u}_1]$, $\boldsymbol{W}_{p3} = [\,\boldsymbol{0};\ \boldsymbol{u}_2]$, or even $\boldsymbol{W}_{p2} = [\,\boldsymbol{u}_1;\ \boldsymbol{u}_1]$, $\boldsymbol{W}_{p3} = [\,\boldsymbol{u}_2;\ \boldsymbol{u}_2]$ would also do the job.

Figure 9 shows the network in action when item Sarah is presented to the initialized network (all states zero). Each column of figure 9 corresponds to a certain time slice (discrete time progresses from left to right), while the *i*-th row shows state progression of *Neurotron i*. Only *Neurotron 1* decodes a *stimulation* (blue) when Sarah is presented, which leads after two steps to *excitation* (violet) and *bursting* (orange), finally to *activation* (red). Since the other two *Neurotrons* detect no stimulus, a stimulus pattern $\boldsymbol{u}_1 = (1,0,0)$ ends with a context $\boldsymbol{c} = \boldsymbol{u}_1 = (1,0,0)$ in phase "burst".

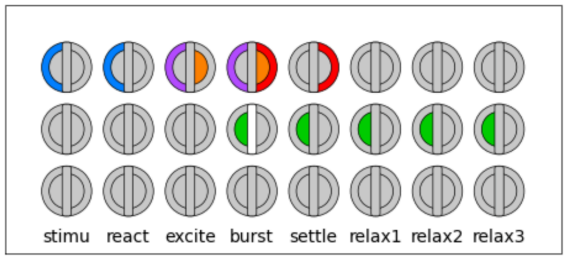

Figure 9: network state transition after presentation of first sequence item Sarah

The second row of figure 9 shows the state transition of *Neurotron 2*. Without stimulus there is neither excitation, bursting or activation. Once *Neurotron 1*, however, activates its output (column "burst") the *prediction terminal* of *Neurotron 2* detects context $\boldsymbol{c} = \boldsymbol{u}_1$ (Sarah) and becomes predictive (green). The white vertical bar symbolizes a *predictive spike*, which mimics an NMDA-spike in neurobiology, depolarizing the soma [2,3,4]. Since the pulse unit of the predictive state is parametrized with a long duty time (figure 4, duty = 8), the predictive state will last until the next sequence item is presented. Neurotron 3 stays passive all the time, since it neither decodes a stimulus, nor does its prediction terminal detect a pattern to trigger.

Figure 10 shows the state transition progress when item loves (token $\boldsymbol{f}_1$) is presented to the network. In this case only Neurotron 2 decodes a stimulus. Since *Neurotron 2* has been set predictive (green) in the previous phase, the stimulus will immediately set the output active (red) in phase "react". A combination of *prediction* and *activation* can be interpreted as a correct prediction, which (in case of an *augmented Neurotron*) would trigger synaptic *learning* in the *prediction* terminal, which is indicated by the yellow vertical bar.

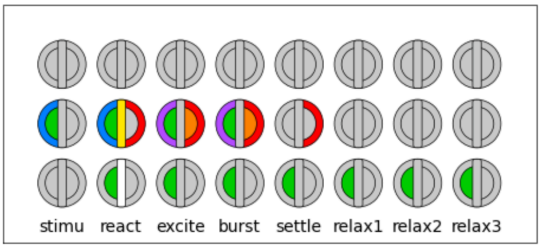

Figure 10: network state transition after presentation of second sequence item loves

In subsequent iterations (phases "excite", "burst") *Neurotron 2* gets excited and bursting, which, however, has no consequence for the network output since the output of *Neurotron 2* has been already activated in phase "react". *Neurotron 1* stays passive (no decoded stimulus, no detected prediction pattern), while in phase "react" (when *Neurotron 2* activates its output) *Neurotron 3* gets predictive (predicting item music), when it detects context $\boldsymbol{c} = \boldsymbol{u}_2 = (0,1,0)$ encoding loves.

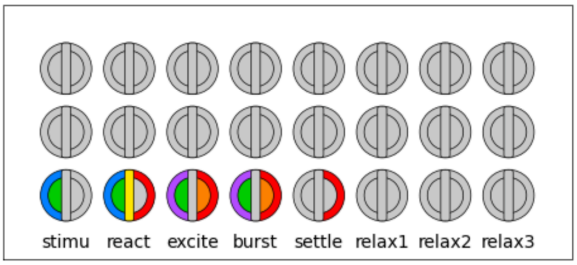

Figure 11: network state transition after presentation of third (and last) sequence item music

Finally, figure 11 shows the state transitions of the network after presentation of music, the last item of the sequence. While Neurotrons 1 and 2 stay passive Neurotron 3 runs a similar state transition pattern like Neurotron 2 in the previous cycle.

References

[1] Rosenblatt, F.: „The perceptron. A probabilistic model for information storage and organization in the brain“; *Psychological Reviews*, 65 (1958): S. 386–408.

[2] Major, G., Larkum, M. E., and Schiller, J.: “Active properties of neocortical pyramidal neuron dendrites”; *Annual Review of Neuroscience* 36, 1–24 (2013).

[3] Antic, S. D., et al.: “The decade of the dendritic NMDA spike”; *J. Neurosci. Res.* 88, 2991–3001. doi: 10.1002/jnr.22444 (2010).

[4] Hawkins J., Ahmad S.: „Why Neurons Have Thousands of Synapses, a Theory of Sequence Memory in Neocortex“; *frontiers in Neural Circuits (2016)*.

[5] Hawkins J., Ahmad S., Cui Y.: „A Theory of How Columns in the Neocortex Enable Learning the Structure of the World“; *frontiers in Neural Circuits (2017)*.

[6] Chavlis S., et all: Dendrites of Dentate Gyrus Granule Cells Contribute to Pattern Separation by Controlling Sparsity. *Hippocampus 27:89-110 (2017).*

[7] Gidon A., et all: Dendritic Action Potentials and Computation in Human Layer 2/3 Cortical Neurons. Science 367, 83-87 (2020).

[8] Magee J.C., Grienberger C.: Synaptic Plasticity Forms and Functions. *Annual Review of Neuroscience 46, 95-117 (2020).*

[9] McCulloch W.S., Pitts W.: A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics,* 5, 115-133 (1943).

[10] Winding M., et all: The connectome of an insect brain. *Science 379, eadd9330* (2023).

[11] Larkum M.E., et all: Synaptic integration in tuft dendrites of layer 5 pyramidal neurons: a new unifying principle. *Science 325:756-60* (2009).

[12] Gidon A., et all: Dendritic Action Potentials and Computation in Human Layer 2/3 Cortical Neurons. *Science 367, 83-87 (2020)*

[13] Major G., et all: Spatiotemporally Graded NMDA Spike/Plateau Potentials in Basal Dendrites of Neocortical Pyramidal Neurons. *Journal of Neurophysiology (Junie, 2008)*

[14] Brette R., Gerstner W.: Adaptive Exponential Integrate-and-Fire Model as an Effective Description of Neural Activity.

[8] Chklovskii et al.: Cortical rewiring and information storage. *Nature* 431, 782–788. doi: 10.1038/nature03012 (2004).

[1] Hawkins J., Blakeslee S.: “On Intelligence”; Owl Books (2005).

[2] Hawkins J.: “A Thousand Brains”; Basic Books, New York (2022).

[9] Stuart, G.J., Häusser, M.:. Dendritic coincidence detection of EPSPs and action potentials. *Nat. Neurosci.* 4, 63–71. doi: 10.1038/82910 (2001)

[10] Berlyand L., Jabin P.E.: „Mathematics of Deep Learning; An Introduction“; *de Gruyter Textbook, 2023*.